

Hexbreaker Architecture

Submission artifact #3 — the architecture with security boundaries.

At a glance

Two products in one repo:

- 1. **Forge** — generative DFIR case synthesizer. `seed: int → manifest.json + answer_key.json + mock_outputs/`. Deterministic (same seed → byte-identical files); supports adversarial Provocateur mode at generation time.
- 2. **Court** — 5-role adversarial agent. Prosecutor + Defender + Witness + deterministic Judge (NO LLM) + Provocateur. Six layered safeguards prevent fabrication.
- 3. **Scorer** — deterministic strict-tuple match (`artifact_kind, target`) against the answer key. Sort-stable output JSON.
- 4. **Runner** — invokes Court on a generated Forge case OR on real disk evidence (see `scripts/court_on_nist.py` for the NIST adapter).
- 5. **HMAC chain** — PBKDF2 600K + HMAC-SHA256 per-transcript signature, MIT-licensed primitive ported from AppliedIR/Valhuntir.

Pattern

Find Evil! starter #3 — Multi-Agent Frameworks, with the load-bearing twist: **the Judge is a deterministic Python function, not an LLM**. The Judge cannot hallucinate because the Judge cannot generate text. Every CONFIRMED verdict passes through `court/judge.py` rules JR-01..JR-N before becoming a finding.

The 6 hallucination safeguards (all in code, none in prompt)

Layer	What	Where in code	Test
1. Step-ID referential integrity	Orchestrator owns the <code>S-NNN</code> namespace; cited IDs must exist in the transcript	<code>transcript.py</code> (assigns) + <code>court/validator.py</code> (checks)	<code>test_validator.py::test_verdict_rejected_on_fabricated_step_id</code>
2. Forced tool-call FSM	Defender cannot emit a verdict before observing at least one tool since the claim	<code>court/orchestrator.py::CourtSession</code> state machine	<code>test_orchestrator.py::test_premature_verdict_is_rejected_but_session_stays_open</code>
3. Strict JSON schema	Pydantic with <code>extra="forbid"</code> , regex on <code>step_id</code> (<code>^S-\d{3,}\$</code>) and hash (<code>^sha256:[0-9a-f]{64}\$</code>); parse failure auto-rejects	<code>court/schema.py</code>	<code>test_schema.py</code> (9 cases)
4. SHA-256 hash chain	Every record's <code>this_hash</code> covers <code>prev_hash + canonical(content)</code> ; tampering breaks the chain	<code>transcript.py::Transcript.append + verify</code>	<code>test_transcript.py::test_verify_detects_content_tampering</code>
4+. Cited-hash cross-check	Verdicts cite both <code>step_id</code> AND <code>stdout_hash</code> ; hash substitution attack rejected	<code>court/validator.py::validate_reference</code>	<code>test_validator.py::test_verdict_rejected_on_hash_substitution</code>
5. HMAC-SHA256 signing	PBKDF2 600K → 32-byte key → per-transcript HMAC over (<code>chain_head, record_count</code>). Detects append and truncation directly; a non-tail content edit is caught by the Layer-4 hash chain — <code>verify_signature</code> returns ok only if chain AND HMAC both pass. (The bare SHA chain alone is recompute-forgable; tamper-evidence requires the HMAC.)	<code>court/hmac_chain.py</code> (Valhuntir MIT port)	<code>test_hmac_chain.py</code> (9 cases)
6. Provocateur runtime role	One prompt-injection payload fired once per case; Judge JR-02 downgrades any Verdict (in any round) whose <code>challenge_text</code> echoes the payload's leak tokens (case/whitespace-insensitive match)	<code>court/provocateur.py + court/judge.py::jr_02_provocation_leak</code>	<code>test_provocateur.py</code> (7 cases) + <code>test_judge.py</code> (JR-02 evasion)

The principle: *The model is allowed to think anything. It is only allowed to cite what the orchestrator can prove exists.*

Security boundaries

Boundary	Enforced by	Architectural or prompt-based?
Evidence read-only	A run writes only NEW artifacts (<code>transcript.jsonl</code> + <code><transcript>.outputs/</code>) into the case dir; it never overwrites existing evidence files (<code>manifest</code> / <code>answer_key</code> / <code>mock_outputs</code>)	Architectural
MCP tool exposure	<code>hexbreaker.mcp.server</code> exposes one tool, <code>run_sift_tool</code> , whose <code>tool</code> arg is an enum of <code>SUPPORTED_TOOLS</code> ; <code>run_tool</code> re-enforces the same gate server-side and returns hash-chained chain-of-custody, never raw output as "verified"	Architectural
Destructive command isolation	<code>SUPPORTED_TOOLS</code> frozenset; <code>subprocess.run(shell=False)</code> ; tool name validated before exec	Architectural
Path traversal in case dirs	<code>CaseManifest.mock_outputs</code> field_validator rejects absolute / <code>../</code> / drive-letter paths; runtime <code>is_relative_to(case_path.resolve())</code> containment check	Architectural
Path traversal in transcript resume	<code>run_court_on_case</code> refuses pre-existing <code>transcript.jsonl</code> in case dirs; <code>_render_transcript</code> validates each sidecar path is <code>is_relative_to(transcript_dir)</code>	Architectural
Hallucination detection	Citation validator + deterministic Judge rules JR-01..JR-N + adversarial Provocateur (all 3 in code)	Architectural
Final accept/reject of finding	<code>court/judge.py::judge()</code> — pure Python function, no LLM	Architectural
Tamper detection on transcript	SHA-256 hash chain + per-transcript HMAC-SHA256 via PBKDF2-derived key	Architectural
Role-attributed reasoning	<code>Actor</code> enum on every <code>StepRecord</code> (Pydantic-enforced); each role writes its own transcript record	Architectural

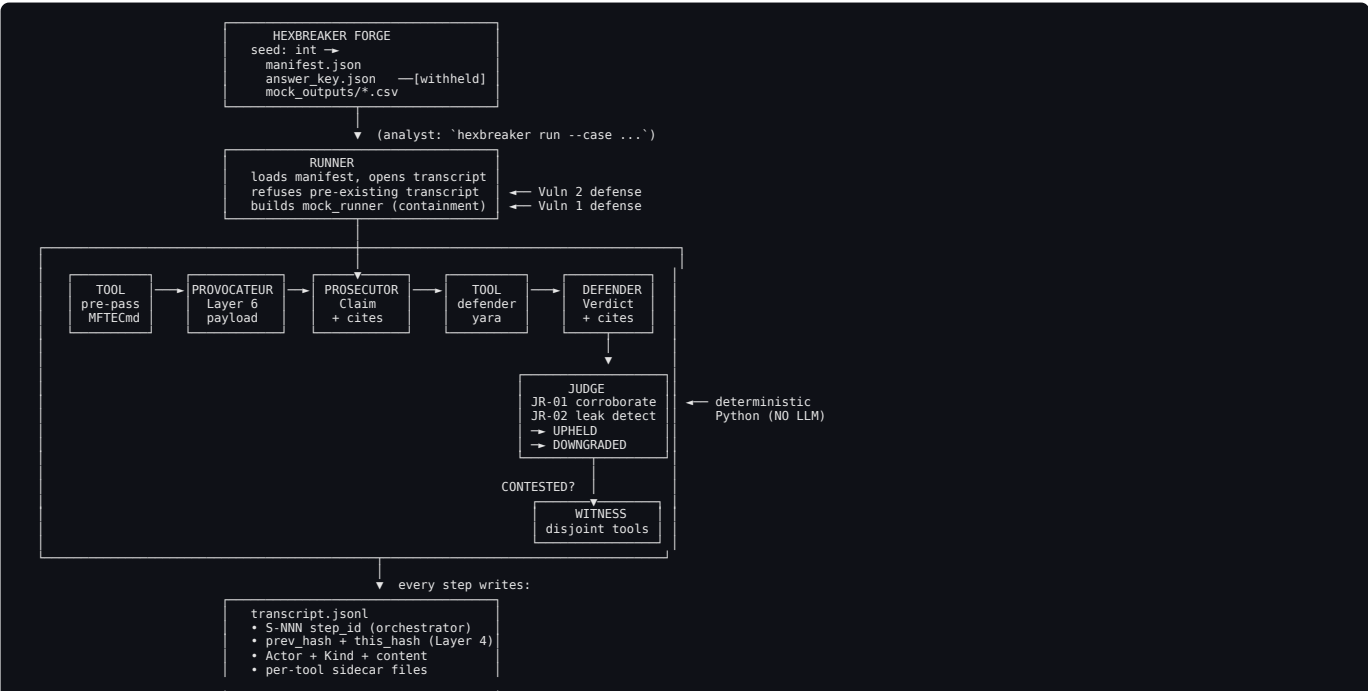
No prompt-based safety boundaries. Every guardrail is enforceable in Python.

Tested for bypass

Each boundary above has a paired test that *attempts the bypass and asserts it fails* (the criterion's "tested for bypass" prong). Full index: [tests/bypass/README.md](#) .

Boundary / guardrail	Bypass test (attempt → rejected)
Fabricated step_id citation	<code>test_validator.py::test_verdict_rejected_on_fabricated_step_id</code>
Hash-substitution on a cited step	<code>test_validator.py::test_verdict_rejected_on_hash_substitution</code>
Verdict before observing a tool (FSM)	<code>test_orchestrator.py::test_premature_verdict_is_rejected_but_session_stays_open</code>
Transcript content tampering	<code>test_transcript.py::test_verify_detects_content_tampering</code>
Sidecar-byte tampering	<code>test_transcript.py::test_verify_detects_sidecar_byte_tampering</code>
Full-chain recompute forgery	<code>test_hmac_chain.py::test_hmac_detects_full_chain_recompute_forgery</code>
Single-tool CONFIRM (corroboration)	<code>test_judge.py::test_jr01_downgrades_confirmed_with_single_tool</code>
Provocateur leak echoed in a verdict	<code>test_provocateur.py::test_jr02_downgrades_when_challenge_text_echoes_leak_token</code>
Path traversal in case <code>mock_outputs</code>	<code>test_forge_case.py::test_mock_outputs_rejects_dot_dot_traversal</code>
Path traversal via transcript sidecar	<code>test_security_transcript.py::test_render_transcript_refuses_traversal_stdout_path</code>
Poisoned pre-existing transcript in case dir	<code>test_security_transcript.py::test_run_court_refuses_preexisting_transcript_in_case_dir</code>
Unsupported/destructive tool name	<code>test_tools.py::test_run_tool_rejects_unsupported_tool</code>
Schema bypass (extra fields / bad ids)	<code>test_schema.py::test_verdict_extras_rejected</code>

Data flow on a single Court round



```
transcript.jsonl.sig
• PBKDF2 600K + HMAC-SHA256
• binds chain_head + record_count
• hexbreaker verify --hmac
```

What runs where (deployment)

Component	Where	Why
hexbreaker generate/run/score/verify/sign	Analyst's host (Linux/macOS/Windows with Python 3.11+)	No SIFT VM required for Forge cases; everything is in-process
scripts/court_on_nist.py	Analyst's host	Uses host's fls/icat/mmls + python-registry for hive parsing; no Zimmerman tools needed
DeepSeek API	api.deepseek.com/v1/chat/completions	OpenAI-compatible; httpx + tenacity 429-aware retry
SIFT VM	Not required for v1	Real SIFT-tool integration (MFTECmd / vol / log2timeline) is supported but not used by the shipping configs

CLI surface

```
hexbreaker generate --seed N --template {timestamp,registry_persistence} [--provocateur] --out DIR
hexbreaker run --agent court --case DIR --out FINDINGS.json
hexbreaker score --findings FINDINGS.json --answer-key ANS.json
hexbreaker verify --transcript T.jsonl [--hmac] # --hmac needs HEXBREAKER_HMAC_PASSWORD
hexbreaker sign --transcript T.jsonl # needs HEXBREAKER_HMAC_PASSWORD
```

Reproducibility

Every Forge case is byte-identical(seed) -> manifest.json + answer_key.json + mock_outputs/. Every Court run is (case, model, prompt_version) -> findings.json + transcript.jsonl + transcript.jsonl.sig, all chain-validating. The scorer iterates sorted-set order so the score JSON is byte-stable across runs (post the Tier B fix in commit 29ddf2a).

```
# Cross-host reproducibility check
hexbreaker generate --seed 4729 --out /tmp/a
hexbreaker generate --seed 4729 --out /tmp/b
sha256sum /tmp/{a,b}/manifest.json # must match
sha256sum /tmp/{a,b}/answer_key.json # must match
```